

Python Interview Questions

Mutability & References — Top 100

Classes · References · Copying · Internals · Gotchas

Section 1 — Mutable vs Immutable Types

Q1. What does 'mutability' mean in Python?

Mutability describes whether an object's value (internal state) can be changed after it is created. Mutable objects can be modified in-place; immutable objects cannot — any 'change' produces a new object.

```
# Immutable: int
x = 42
print(id(x))    # e.g. 9789344
x += 1
print(id(x))    # different id — new object created

# Mutable: list
lst = [1, 2, 3]
print(id(lst))
lst.append(4)
print(id(lst))  # same id — mutated in-place
```

Q2. Which built-in types are immutable?

int, float, complex, bool, str, bytes, tuple, frozenset, and NoneType. Also range objects and named-tuples are immutable.

```
s = 'hello'
# s[0] = 'H'    # TypeError: 'str' object does not support item assignment

t = (1, 2, 3)
# t[0] = 99     # TypeError

fs = frozenset({1, 2})
# fs.add(3)     # AttributeError
```

Q3. Which built-in types are mutable?

list, dict, set, bytearray, and instances of most user-defined classes are mutable.

```
lst = [1, 2]
lst.append(3)          # in-place

d = {'a': 1}
d['b'] = 2             # in-place

s = {1, 2}
s.add(3)               # in-place

ba = bytearray(b'hi')
ba[0] = 72             # in-place
```

Q4. Why are strings immutable in Python?

Immutability allows strings to be safely shared (interned), used as dict keys, and makes them hashable. It also simplifies thread safety — no lock is needed to read a string concurrently.

```
a = 'hello'
b = 'hello'
print(a is b)    # True on CPython – string interning

# Safe as dict key
d = {'hello': 1}
print(d['hello']) # 1
```

Q5. Can a tuple containing a list be considered immutable?

The tuple itself is immutable (you cannot add/remove/replace elements), but if an element is mutable (like a list), that element can still be mutated. The tuple's identity of its elements stays fixed; their internal state may not.

```
t = ([1, 2], 3)
# t[0] = [] # TypeError – cannot rebind tuple slot
t[0].append(99) # OK – mutates the list inside
print(t)        # ([1, 2, 99], 3)
```

Q6. What is object identity vs equality?

Identity (`is`) checks whether two names point to the exact same object in memory (same id). Equality (`==`) checks whether two objects have the same value, delegating to `\_\_eq\_\_`.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b) # True – same value
print(a is b) # False – different objects
print(a is c) # True – same object
```

```
print(id(a), id(c)) # identical
```

Q7. What does `id()` return?

`id()` returns an integer that is guaranteed to be unique and constant for an object during its lifetime. In CPython it is the memory address of the object.

```
x = 'python'
print(id(x))          # e.g. 140234567

y = x
print(id(x) == id(y)) # True – same object

z = 'python'          # may or may not be interned
print(x is z)         # True on CPython (string interning)
```

Q8. Why should you never use a mutable default argument?

Default argument values are evaluated once at function definition time, not each call. A mutable default is shared across all calls, leading to state leaking between invocations.

```
# BUG
def append_to(item, lst=[]):
    lst.append(item)
    return lst

print(append_to(1)) # [1]
print(append_to(2)) # [1, 2] -- surprise!

# FIX
def append_to_safe(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst
```

Q9. What is the difference between `==` and `is`?

`==` calls `__eq__` and compares values. `is` compares object identity (`id`). Always use `==` for value comparison; use `is` only for singletons like `None`, `True`, `False`.

```
print(1 == 1.0)    # True – equal values
print(1 is 1.0)    # False – different objects, different types

x = None
print(x is None)   # True – correct idiom
print(x == None)   # True – but fragile (__eq__ could be overridden)
```

Q10. What is integer caching / small integer caching in CPython?

CPython pre-allocates integer objects for values in `[-5, 256]`. References to these values always return the same object, so ``is`` returns `True` for small integers but not for larger ones.

```
a = 256
b = 256
print(a is b)    # True  - cached

c = 257
d = 257
print(c is d)    # False - not cached (implementation detail)

# Never rely on this - use == for value comparison
```

Q11. What is string interning?

CPython automatically interns string literals that look like identifiers (no spaces, all alphanumeric). Interned strings share the same object. You can force interning with ``sys.intern()``.

```
import sys

a = 'hello'
b = 'hello'
print(a is b)          # True - auto-interned

c = 'hello world'
d = 'hello world'
print(c is d)          # False - space prevents auto-interning

e = sys.intern('hello world')
f = sys.intern('hello world')
print(e is f)          # True
```

Q12. How does Python manage memory for immutable objects?

Immutable objects can be safely shared. CPython uses reference counting plus a cyclic garbage collector. Small ints and interned strings are cached globally; other immutables are freed when their reference count drops to zero.

```
import sys

x = 'hello'
print(sys.getrefcount(x))  # 2 (x + getrefcount arg)

y = x                      # ref count increases
print(sys.getrefcount(x))  # 3
```

Q13. Can ``frozenset`` be used as a dict key? Why not a regular ``set``?

frozenset is immutable and therefore hashable — it implements `__hash__`. A regular set is mutable and explicitly sets `__hash__ = None`, making it unhashable.

```
fs = frozenset({1, 2, 3})
d = {fs: 'value'}
print(d[fs])    # 'value'

s = {1, 2, 3}
# d2 = {s: 'x'}  # TypeError: unhashable type: 'set'
```

Q14. What happens when you 'modify' an immutable object?

A new object is created with the new value, and the variable is rebound to point at it. The original object is unchanged and may be garbage-collected if unreferenced.

```
x = 'hello'
old_id = id(x)
x = x + ' world'    # creates new str object
print(id(x) == old_id)  # False

n = 10
old_id = id(n)
n += 1
print(id(n) == old_id)  # False (outside small-int cache)
```

Q15. How does mutability affect hashing?

Only hashable objects can be used as dict keys or set members. An object is hashable if it has a `__hash__` method that returns a stable integer. Mutable objects are generally unhashable because their hash would change as their contents change, breaking lookup invariants.

```
print(hash('hello'))    # stable
print(hash((1, 2, 3)))  # stable

# print(hash([1, 2, 3]))    # TypeError
# print(hash({'a': 1}))     # TypeError

# Tuple with mutable element is also unhashable
# print(hash([1, 2], 3))    # TypeError
```

Q16. What is `__hash__` and when must you define it?

If you define `__eq__` on a class, Python sets `__hash__ = None` (unhashable). You must also define `__hash__` returning a stable integer if you want instances usable in sets/dicts.

```
class Point:
    def __init__(self, x, y): self.x, self.y = x, y
    def __eq__(self, other): return (self.x, self.y) == (other.x, other.y)
    def __hash__(self): return hash((self.x, self.y))
```

```
s = {Point(1,2), Point(1,2), Point(3,4)}
print(len(s))    # 2
```

Q17. Why is `None` used as a sentinel instead of `0` or `False`?

`None` is a singleton (there is exactly one `None` object) and is identity-checkable with `is None`. Using `0` or `False` as sentinels fails when those are valid values in the domain.

```
def find(lst, target):
    for i, v in enumerate(lst):
        if v == target:
            return i
    return None    # not 0 or -1, which could be valid indices

result = find([10, 20], 20)
if result is not None:
    print('Found at index', result)    # Found at index 1
```

Q18. What is the `__slots__` impact on mutability?

`__slots__` restricts which attributes exist, but the listed attributes are still mutable unless you use descriptors or properties to prevent writes.

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Point(1, 2)
p.x = 99    # allowed — __slots__ doesn't freeze values
# p.z = 0    # AttributeError — z not in __slots__
```

Q19. How do you make a truly immutable custom class?

Override `__setattr__` and `__delattr__` to raise `AttributeError`, and use `object.__setattr__` in `__init__` to bypass your own hook during initialisation.

```
class Frozen:
    def __init__(self, x, y):
        object.__setattr__(self, 'x', x)
        object.__setattr__(self, 'y', y)
    def __setattr__(self, name, value):
        raise AttributeError('Frozen is immutable')
    def __delattr__(self, name):
        raise AttributeError('Frozen is immutable')

f = Frozen(1, 2)
print(f.x)    # 1
# f.x = 99    # AttributeError
```

Q20. What is a NamedTuple and how does it differ from a regular tuple?

A NamedTuple is an immutable, ordered collection whose fields are accessible by name as well as by index. It inherits all tuple properties (immutability, hashability) while adding readability.

```
from collections import namedtuple
from typing import NamedTuple

# Classic style
Point = namedtuple('Point', ['x', 'y'])
p = Point(1, 2)
print(p.x, p[0])    # 1 1

# Typed style
class Vector(NamedTuple):
    x: float
    y: float

v = Vector(3.0, 4.0)
# v.x = 0    # AttributeError – immutable
```

Section 2 — References & Variable Assignment

Q21. What is a variable in Python? Is it a box or a label?

A variable is a label (name) that references an object. Python does not store values inside variables — objects exist independently in memory and names are bound to them. Multiple names can reference the same object.

```
x = [1, 2, 3]
y = x          # y is another label for the same list
y.append(4)
print(x)       # [1, 2, 3, 4] – x sees the mutation too
```

Q22. What happens during assignment (`a = b`)?

Python binds the name `a` to whatever object `b` currently references. No copy is made. Both names point to the same object.

```
a = [10, 20]
b = a
print(id(a) == id(b))    # True

b = [10, 20]              # rebind b to a NEW list
print(id(a) == id(b))    # False – now separate objects
```

Q23. What is augmented assignment (`+=`) on mutable vs immutable types?

For mutable types, `+=` calls `\_\_iadd\_\_` and mutates in-place (same id). For immutable types, it creates a new object (new id).

```
# Mutable (list)
lst = [1, 2]
old = id(lst)
lst += [3]
print(id(lst) == old)    # True – in-place

# Immutable (tuple)
t = (1, 2)
old = id(t)
t += (3,)
print(id(t) == old)      # False – new object
```

Q24. Explain the surprising tuple `+=` behaviour.

Even though `t += (3,)` on a tuple raises a `TypeError` when `t` is inside another container (because tuples are immutable), the mutation to any mutable element inside CAN have already occurred. This is a known Python quirk.

```
t = ([1, 2], 3)
try:
    t[0] += [4]    # t[0].__iadd__([4]) succeeds (mutates list)
                  # then tuple assignment fails
except TypeError:
    pass
print(t)          # ([1, 2, 4], 3) – list WAS mutated!
```

Q25. What is the difference between shallow copy and deep copy?

A shallow copy creates a new container but populates it with references to the same objects as the original. A deep copy recursively copies all objects, producing fully independent structures.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
shallow[0].append(99)
print(original)    # [[1, 2, 99], [3, 4]] – inner list shared!

original2 = [[1, 2], [3, 4]]
deep = copy.deepcopy(original2)
deep[0].append(99)
print(original2)   # [[1, 2], [3, 4]] – fully independent
```


Q26. How do you make a shallow copy of a list?

Multiple idiomatic ways exist: slice ``[:]``, ``list()``, ``copy.copy()``, or ``copy()`` method.

```
original = [1, 2, 3]

a = original[:]
b = list(original)
c = original.copy()

import copy
d = copy.copy(original)

print(a is original)  # False – new list
print(a == original)  # True  – same values
```

Q27. How do you make a shallow copy of a dict?

Use ``dict.copy()``, ``dict()``, or ``copy.copy()``.

```
original = {'a': [1, 2], 'b': 3}
shallow = original.copy()

shallow['b'] = 99
print(original['b'])  # 3 – independent scalar

shallow['a'].append(99)
print(original['a'])  # [1, 2, 99] – shared inner list!
```

Q28. When does ``copy.deepcopy()`` fail?

`deepcopy` fails when objects contain: open file handles, sockets, threads, locks, database connections, or lambdas that capture closures with non-copyable state. You can customise behaviour via ``__deepcopy__``.

```
import copy, threading

lock = threading.Lock()
try:
    copy.deepcopy(lock)  # TypeError on some Python versions
except Exception as e:
    print(type(e).__name__, e)
```

Q29. What is aliasing?

Aliasing occurs when two or more names reference the same mutable object. A mutation through one name is visible through all others.

```
def add_item(container, item):
    container.append(item)
```

```
my_list = [1, 2]
add_item(my_list, 3)
print(my_list)  # [1, 2, 3] - my_list was aliased inside the fn
```

Q30. How does Python pass arguments to functions (pass-by-value or pass-by-reference)?

Neither exactly. Python uses 'pass-by-object-reference' (also called 'pass-by-assignment'). The function receives a reference to the same object. Rebinding the parameter inside the function does not affect the caller; mutating a mutable object does.

```
def rebind(x):
    x = [99]          # local rebind - caller unaffected

def mutate(x):
    x.append(99)      # mutates the shared object

lst = [1, 2]
rebind(lst)
print(lst)  # [1, 2]

mutate(lst)
print(lst)  # [1, 2, 99]
```

Q31. What is the difference between rebinding and mutating?

Rebinding (`x = new_obj`) makes a name point to a different object; the original object is unchanged. Mutating (`x.append(v)`) changes the object's internal state without changing the name's binding.

```
a = [1, 2, 3]
b = a

# Rebind a
a = [4, 5, 6]
print(b)  # [1, 2, 3] - b still points to original

a = [1, 2, 3]
b = a
# Mutate a
a.append(4)
print(b)  # [1, 2, 3, 4] - b sees mutation
```

Q32. What is reference counting in CPython?

CPython tracks how many names/structures reference each object via a counter. When the count reaches zero, memory is reclaimed immediately. Use `sys.getrefcount()` to inspect (adds 1 for the call itself).

```
import sys

x = []
print(sys.getrefcount(x)) # 2 (x + arg)

y = x
print(sys.getrefcount(x)) # 3

del y
print(sys.getrefcount(x)) # 2 again
```

Q33. What is the cyclic garbage collector and why is it needed?

Reference counting alone cannot free objects that reference each other (cycles). The cyclic GC periodically detects reference cycles and frees them. It is triggered automatically but can be controlled via the ``gc`` module.

```
import gc

class Node:
    def __init__(self): self.ref = None

a = Node()
b = Node()
a.ref = b # cycle
b.ref = a
del a, b # ref counts > 0 – GC needed

gc.collect() # force collection
print('Collected:', gc.collect())
```

Q34. What is ``del`` in Python? Does it free memory immediately?

``del`` removes a name binding (decrements the reference count). Memory is freed immediately only if the reference count reaches zero. If other names still reference the object, it lives on.

```
x = [1, 2, 3]
y = x
del x # removes x, ref count 2->1
print(y) # [1, 2, 3] – still alive via y

del y # ref count 1->0 – freed immediately
```

Q35. How do ``weakref`` references work?

A weak reference does not increment the reference count. If the only remaining reference to an object is weak, the object can be garbage-collected. Use ``weakref.ref()`` or ``weakref.WeakValueDictionary`` to avoid memory leaks in caches.

```
import weakref

class Obj: pass

o = Obj()
wr = weakref.ref(o)

print(wr())          # <Obj object>  - alive
del o
print(wr())          # None           - collected
```

Q36. What are the three strategies for copying an object?

1) Assignment (`b = a`) — alias, no copy. 2) Shallow copy — new container, shared inner objects. 3) Deep copy — fully independent recursive copy.

```
import copy
original = {'data': [1, 2, 3]}

alias    = original
shallow  = copy.copy(original)
deep     = copy.deepcopy(original)

original['data'].append(99)
print(alias['data'])    # [1, 2, 3, 99] - same object
print(shallow['data'])  # [1, 2, 3, 99] - shared inner
print(deep['data'])     # [1, 2, 3]    - independent
```

Q37. How does slicing relate to copying for lists?

A slice like `a[:]` or `a[1:3]` always produces a new list (shallow copy). The list object is new, but the elements inside are the same objects as in the original.

```
original = [[1], [2], [3]]
sliced   = original[: ]

print(sliced is original)    # False - new list
print(sliced[0] is original[0]) # True - shared inner

sliced[0].append(99)
print(original[0])           # [1, 99] - affected
```

Q38. What is the difference between `list.copy()` and `list[:]`?

They produce identical results (shallow copy) in terms of semantics. `list.copy()` is more readable; `list[:]` is a historical idiom. Both create a new list object populated with the same element references.

```
a = [1, 2, 3]
b = a.copy()
c = a[: ]
```

```
print(b == c == a)    # True
print(b is a or c is a) # False – both are new lists
```

Q39. How does `dict.copy()` behave with nested dicts?

It produces a shallow copy. Top-level keys are independent, but nested dicts/lists are still shared.

```
d = {'outer': {'inner': [1, 2]}}
copy = d.copy()

copy['outer']['inner'].append(99)
print(d) # {'outer': {'inner': [1, 2, 99]}} – shared

# Fix: deepcopy
import copy
deep = copy.deepcopy(d)
```

Q40. What is object interning and when does Python use it?

Python reuses (interns) certain immutable objects rather than creating new ones: small integers [-5,256], string literals that look like identifiers, compile-time constant expressions. This saves memory and speeds up identity comparisons.

```
# Small int cache
print(100 is 100)    # True
print(1000 is 1000)  # False (likely – interpreter-dependent)

# Compile-time folding
a = 'abc' * 2        # 'abcbcb' – folded at compile time
b = 'abcbcb'
print(a is b)        # True on CPython
```

Q41. What is the `copy` module and when do you use each function?

`copy.copy()` for shallow copies of containers; `copy.deepcopy()` for fully independent recursive copies. Use shallow copy when inner objects are immutable or intentionally shared; deep copy when you need complete independence.

```
import copy

# Shallow: fine when values are immutable
cfg = {'host': 'localhost', 'port': 8080}
cfg2 = copy.copy(cfg)

# Deep: needed for nested mutable structures
graph = {'nodes': [1, 2], 'edges': [(1,2)]}
graph2 = copy.deepcopy(graph)
```

Q42. How does `**dict` unpacking relate to copying?

`{**d}` creates a new shallow-copy dict. It's equivalent to `d.copy()` but also useful for merging.

```
d = {'a': [1, 2], 'b': 3}
d2 = {**d}

print(d2 is d)           # False – new dict
print(d2['a'] is d['a']) # True – shared inner list
```

Q43. What is the LEGB rule for name resolution?

Python resolves names in order: Local → Enclosing → Global → Built-in. The first match wins.

```
x = 'global'

def outer():
    x = 'enclosing'
    def inner():
        # x = 'local' # if this line existed, local wins
        print(x)      # 'enclosing' – found in enclosing
    inner()

outer() # enclosing
```

Q44. What is the `global` keyword?

`global` declares that a name inside a function refers to the module-level variable, allowing the function to rebind it.

```
count = 0

def increment():
    global count
    count += 1

increment()
increment()
print(count) # 2
```

Q45. What is the `nonlocal` keyword?

`nonlocal` lets an inner function rebind a name from an enclosing (but non-global) scope. Without it, assignment would create a new local variable.

```
def make_counter():
    n = 0
    def increment():
```

```
        nonlocal n
        n += 1
        return n
    return increment

counter = make_counter()
print(counter()) # 1
print(counter()) # 2
```

Section 3 — Common Gotchas & Traps

Q46. What is the 'late binding closures' problem?

Closures in Python bind to variables by reference, not by value. In a loop, all closures capture the same loop variable, which has its final value after the loop ends.

```
fns = [lambda: i for i in range(3)]
print([f() for f in fns]) # [2, 2, 2] - all see i=2

# Fix: use default argument to capture current value
fns = [lambda i=i: i for i in range(3)]
print([f() for f in fns]) # [0, 1, 2]
```

Q47. Why does modifying a list while iterating over it cause issues?

Python's list iterator tracks an integer index. Removing elements shifts items, causing the iterator to skip elements or raise `IndexError`.

```
lst = [1, 2, 3, 4]
for x in lst:
    if x % 2 == 0:
        lst.remove(x) # BUG - skips elements
print(lst) # [1, 3] but [3] is skipped during removal

# Safe: iterate over a copy
lst = [1, 2, 3, 4]
for x in lst[:]:
    if x % 2 == 0:
        lst.remove(x)
print(lst) # [1, 3] - correct
```

Q48. What happens when you modify a dict while iterating over it?

Python raises `RuntimeError: dictionary changed size during iteration`. Always iterate over a copy of keys if you need to mutate.

```

d = {'a': 1, 'b': 2, 'c': 3}

# WRONG
# for k in d:
#     if d[k] < 2: del d[k]    # RuntimeError

# CORRECT
for k in list(d.keys()):
    if d[k] < 2:
        del d[k]
print(d)    # {'b': 2, 'c': 3}

```

Q49. Why can a tuple's hash change if it contains mutable elements?

A tuple's hash is derived from its elements. If an element is mutable (unhashable), the tuple itself is unhashable. If somehow a mutable element's value changed and were hashable, the tuple's hash would differ — violating hash contract.

```

t1 = (1, 2, 3)
print(hash(t1))    # works

t2 = (1, [2, 3])
# print(hash(t2))   # TypeError: unhashable type: 'list'

```

Q50. What is the difference between `list.sort()` and `sorted()`?

`list.sort()` mutates in-place and returns `None`. `sorted()` returns a new list and works on any iterable.

```

a = [3, 1, 2]

result = a.sort()    # in-place mutation
print(a)             # [1, 2, 3]
print(result)        # None

b = [3, 1, 2]
result2 = sorted(b)  # new list
print(b)             # [3, 1, 2] — unchanged
print(result2)       # [1, 2, 3]

```

Q51. Why does `a = b = []` create one list, not two?

Chained assignment binds all names to the same object. `a` and `b` are aliases of one list.

```

a = b = []
print(a is b)    # True
a.append(1)
print(b)         # [1] — same object

# If you want independent lists:
a, b = [], []

```



```
print(a is b)    # False
```

Q52. Why does multiplying a list of lists with `\*` create shared rows?

`[[0] \* n] \* n` creates n references to the same inner list, not n independent lists.

```
# WRONG – all rows are the same object
matrix = [[0] * 3] * 3
matrix[0][1] = 99
print(matrix)    # [[0, 99, 0], [0, 99, 0], [0, 99, 0]]

# CORRECT – list comprehension creates independent rows
matrix = [[0] * 3 for _ in range(3)]
matrix[0][1] = 99
print(matrix)    # [[0, 99, 0], [0, 0, 0], [0, 0, 0]]
```

Q53. What does `copy.deepcopy()` do with circular references?

deepcopy maintains a memo dict mapping `id(original)` to the copied object. When a circular reference is encountered, it returns the already-copied object from memo, preventing infinite recursion.

```
import copy

a = []
a.append(a)    # circular: a refers to itself

b = copy.deepcopy(a)    # works – memo breaks the cycle
print(b[0] is b)        # True – circularity preserved
```

Q54. What is the difference between `is None` and `== None`?

`is None` is always correct for None-checking because None is a singleton. `== None` invokes `\_\_eq\_\_` which could be overridden and return unexpected results.

```
class Weird:
    def __eq__(self, other): return True    # == everything

w = Weird()
print(w == None)    # True – dangerous!
print(w is None)    # False – correct
```

Q55. What is the `\_\_copy\_\_` / `\_\_deepcopy\_\_` protocol?

Define `\_\_copy\_\_(self)` to customise shallow copy and `\_\_deepcopy\_\_(self, memo)` to customise deep copy. The memo dict prevents infinite recursion for circular structures.

```

import copy

class Config:
    def __init__(self, data, secret):
        self.data = data
        self.secret = secret

    def __copy__(self):
        return Config(self.data, self.secret)

    def __deepcopy__(self, memo):
        return Config(copy.deepcopy(self.data, memo), '***')

c = Config([1, 2], 'pw')
d = copy.deepcopy(c)
print(d.secret)    # ***

```

Q56. What is the effect of returning a mutable object from a function?

The caller receives a reference to the same mutable object. If the caller mutates it, the internal state may change unexpectedly. Return a copy when you want to protect internal state.

```

class Bag:
    def __init__(self): self._items = []
    def add(self, x): self._items.append(x)
    def items(self): return self._items # exposes internals!

b = Bag()
b.add(1)
exposed = b.items()
exposed.append(99)          # mutates Bag internals
print(b.items())            # [1, 99]

# Fix: return a copy
# def items(self): return list(self._items)

```

Q57. What is a 'frozen' dataclass?

Setting `frozen=True` on `@dataclass` makes instances immutable: `\_\_setattr\_\_` and `\_\_delattr\_\_` are overridden to raise `FrozenInstanceError`, and `\_\_hash\_\_` is generated automatically.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class Point:
    x: float
    y: float

p = Point(1.0, 2.0)
# p.x = 99    # FrozenInstanceError
print(hash(p)) # hashable
d = {p: 'origin'}

```

Q58. Why is appending to a list inside a closure tricky?

The closure captures the list name, not a snapshot. If the outer function reassigns the name before the closure runs, the closure sees the new object.

```
def make():
    results = []
    def add(x): results.append(x)    # captures 'results' name
    return add, results

add, res = make()
add(1); add(2)
print(res)    # [1, 2] - works as expected here

# But beware reassignment in the outer scope changing the binding
```

Q59. What is the difference between `extend` and `+=` for lists?

Both mutate in-place and are semantically equivalent for lists. `+=` calls `\_\_iadd\_\_` (which calls `extend` internally). Both return `None` from `extend` but `+=` rebinds the variable.

```
a = [1, 2]
b = a
a += [3]
print(a is b)    # True - same list, in-place

a = [1, 2]
b = a
a.extend([3])
print(a is b)    # True - same list
```

Q60. What is 'copy-on-write' and does Python use it?

Copy-on-write (CoW) delays copying a resource until it is modified. Python does NOT use CoW for its built-in containers. However, the OS uses CoW for `fork()`, and some libraries (e.g., pandas with Arrow backend, PyPy) implement it internally.

```
# Python lists never use CoW - always eager shared references
a = [1, 2, 3]
b = a            # alias, no copy at all
b.append(4)      # mutates the shared object immediately
print(a)         # [1, 2, 3, 4]
```

Q61. What happens to default mutable argument if you reassign it inside the function?

Reassignment creates a new local binding. The default object is not modified. Only mutation (append, update, etc.) affects the default.

```
def fn(lst=[]):
    lst = lst + [1]    # creates new list – default untouched
    return lst

print(fn())    # [1]
print(fn())    # [1] – default still []

def fn2(lst=[]):
    lst.append(1)      # mutates default!
    return lst

print(fn2())    # [1]
print(fn2())    # [1, 1] – accumulates
```

Q62. How does `set` handle mutable objects?

Sets require hashable elements. Mutable objects (list, dict, set) are unhashable and cannot be stored in a set. Use frozenset or tuples as set elements.

```
# OK
s = {1, 'hello', (1, 2), frozenset({3})}

# ERRORS
# s.add([1,2])          # TypeError: unhashable type: 'list'
# s.add({'a': 1})        # TypeError: unhashable type: 'dict'
```

Q63. How does mutability affect multiprocessing vs threading?

Threads share memory, so mutable shared state needs locks. Processes have separate memory spaces, so a mutable object must be explicitly shared via `multiprocessing.Value`, `Array`, or `Manager` proxies.

```
from multiprocessing import Manager, Process

def worker(shared_list, item):
    shared_list.append(item)

with Manager() as mgr:
    lst = mgr.list()
    p = Process(target=worker, args=(lst, 42))
    p.start(); p.join()
    print(list(lst))    # [42]
```

Q64. What is the difference between `bytes` and `bytearray`?

`bytes` is immutable — once created, individual bytes cannot be changed. `bytearray` is mutable — you can modify individual bytes in-place. Both are sequences of integers 0-255.

```
b = bytes([72, 101, 108])
# b[0] = 104    # TypeError
```

```
ba = bytearray([72, 101, 108])
ba[0] = 104      # OK
print(ba)        # bytearray(b'hel')

# bytes is hashable; bytearray is not
print(hash(b))   # works
# print(hash(ba)) # TypeError
```

Q65. What is the `memoryview` object and mutability?

A `memoryview` gives a view into a buffer (bytes, bytearray, array) without copying. If the underlying object is mutable (bytearray), the view allows in-place edits.

```
ba = bytearray(b'hello')
mv = memoryview(ba)
mv[0] = 72          # modifies ba directly
print(ba)           # bytearray(b'Hello')

b = bytes(b'hello')
mv2 = memoryview(b) # read-only view
# mv2[0] = 72       # TypeError
```

Section 4 — Advanced, Internals & Patterns

Q66. How does Python implement list internally?

CPython's list is a dynamic array of object pointers (PyObject*). When capacity is exceeded it over-allocates (roughly 1.125x) to amortise $O(1)$ append. Insertion/deletion in the middle is $O(n)$ due to shifting.

```
import sys

lst = []
prev = sys.getsizeof(lst)
for i in range(10):
    lst.append(i)
    size = sys.getsizeof(lst)
    if size != prev:
        print(f'len={len(lst)}, size={size}') # jumps at realloc
        prev = size
```

Q67. How does Python implement dict internally?

Since Python 3.7 dict uses a compact hash table with insertion-order preservation. Keys are hashed; collisions resolved by open addressing. Load factor threshold triggers resize at $\sim 2/3$ full.

```
d = {}
for i in range(5):
    d[i] = i * i

# Insertion order preserved (Python 3.7+)
print(list(d.keys())) # [0, 1, 2, 3, 4]

# Hash table probing
print(hash('key') % 8) # slot index in initial table of 8
```

Q68. What is the time complexity of common list operations?

append: $O(1)$ amortised, insert(0, x): $O(n)$, pop(): $O(1)$, pop(0): $O(n)$, index lookup: $O(1)$, search (in): $O(n)$, sort: $O(n \log n)$.

```
from collections import deque
import timeit

# Prefer deque for  $O(1)$  front operations
dq = deque([1, 2, 3])
dq.appendleft(0) #  $O(1)$ 
dq.popleft()     #  $O(1)$ 

lst = [1, 2, 3]
lst.insert(0, 0) #  $O(n)$  – shifts all elements
```

Q69. What is the time complexity of dict operations?

get/set/delete: $O(1)$ average, $O(n)$ worst-case due to hash collisions. Iteration: $O(n)$. Collision quality depends on `__hash__` distribution.

```
d = {i: i for i in range(10**6)}

import timeit
print(timeit.timeit(lambda: d[500000], number=10**6)) # fast  $O(1)$ 
```

Q70. How does Python's garbage collector handle `__del__`?

Objects with `__del__` that are part of a reference cycle used to be uncollectable (Python < 3.4). Since Python 3.4 (PEP 442), the GC calls `__del__` finalizers for objects in cycles before reclaiming memory.

```
import gc

class Node:
    def __del__(self):
        print(f'Node {id(self)} collected')

a = Node(); b = Node()
a.b = b; b.a = a # cycle
del a, b
```

```
gc.collect()          # prints both 'collected' messages
```

Q71. What is the difference between `copy.copy` and `object.__copy__`?

`copy.copy(obj)` checks for `__copy__` method first; if found, calls it. Otherwise it falls back to protocol-based shallow copying. Defining `__copy__` gives you full control over what 'shallow copy' means for your class.

```
import copy

class Pair:
    def __init__(self, a, b): self.a, self.b = a, b
    def __copy__(self): return Pair(self.a, self.b)

p = Pair([1], [2])
q = copy.copy(p)
print(q.a is p.a)    # True – shallow: same inner lists
```

Q72. How do you implement a copy constructor pattern in Python?

Accept an optional existing instance and copy its state, or use `classmethod` as a named factory.

```
class Config:
    def __init__(self, host='localhost', port=80):
        self.host = host
        self.port = port

    @classmethod
    def from_config(cls, other):
        return cls(other.host, other.port)

c1 = Config('example.com', 443)
c2 = Config.from_config(c1)
c2.port = 80
print(c1.port, c2.port)    # 443 80 – independent
```

Q73. How do generators relate to mutability and references?

Generator objects are mutable (they maintain internal state). Yielded values are references to objects — the same aliasing rules apply.

```
def gen(data):
    for item in data:
        yield item

lst = [[1], [2], [3]]
g = gen(lst)
first = next(g)    # reference to lst[0]
first.append(99)
print(lst[0])      # [1, 99] – mutation visible
```

Q74. What happens with mutable objects in sets after modification?

Sets use hash values to place elements. If you could somehow mutate an element's hash-relevant state after insertion, the set's internal structure would be corrupted. Python prevents this by disallowing unhashable (mutable) objects in sets.

```
# You can't add mutable objects – enforced at insertion
s = set()
t = (1, 2)      # immutable, hashable
s.add(t)
print(s)        # {(1, 2)}

# t[0] = 99      # TypeError – tuple is immutable, so hash is stable
```

Q75. What is `\_\_reduce\_\_` and how does it relate to copying?

`\_\_reduce\_\_` is the pickle serialization protocol. `copy.deepcopy` uses pickle internally for objects without `\_\_deepcopy\_\_`. Defining `\_\_reduce\_\_` correctly ensures both pickling and deep copying work.

```
import copy

class Point:
    def __init__(self, x, y): self.x, self.y = x, y
    def __reduce__(self): return (Point, (self.x, self.y))

p = Point(1, 2)
q = copy.deepcopy(p)
print(q.x, q.y)    # 1 2
print(q is p)      # False
```

Q76. How do you share state safely between threads with mutable objects?

Use threading primitives: `threading.Lock` for mutual exclusion, `threading.RLock` for reentrant locks, `queue.Queue` for producer-consumer. Avoid sharing mutable state when possible.

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    with lock:
        counter += 1

threads = [threading.Thread(target=increment) for _ in range(1000)]
[t.start() for t in threads]
[t.join() for t in threads]
print(counter)    # 1000 – correct with lock
```

Q77. What is a `NameError` vs `UnboundLocalError` in the context of references?

`NameError` occurs when a name has never been defined in any accessible scope.

`UnboundLocalError` (a subclass) occurs when a name exists in local scope (due to any assignment in the function) but is used before being assigned.

```
def bad():
    print(x)      # UnboundLocalError – x is local (line below)
    x = 10        # presence of this makes x local throughout fn

# print(no_such_var) # NameError – never defined
```

Q78. What is the difference between `vars()`, `locals()`, and `globals()`?

`vars(obj)` returns `obj.__dict__`. `locals()` returns a snapshot of the current local scope (copy — modifying it does NOT affect real locals). `globals()` returns the actual module global dict (modifying it DOES take effect).

```
x = 42
g = globals()
g['x'] = 99
print(x)      # 99 – globals() is live

def fn():
    y = 10
    l = locals()
    l['y'] = 99 # no effect on actual y
    print(y)   # 10
fn()
```

Q79. What is interning and how can you control it?

You can force string interning with `sys.intern()`. This is useful for large sets of repeated strings (e.g., parsing) to reduce memory and allow $O(1)$ identity comparison instead of $O(n)$ equality.

```
import sys

tokens = ['SELECT', 'FROM', 'WHERE'] * 10000
# Without interning: many duplicate string objects

interned = [sys.intern(t) for t in tokens]
# Now all 'SELECT' tokens share one object
print(interned[0] is interned[3]) # True
```

Q80. How does Python's `array` module differ from a list in terms of mutability?

`array.array` is a mutable, typed, compact array (C-style). Elements must be of a declared type (e.g., 'd' for double). It is more memory-efficient than a list for homogeneous numeric data.

```
import array

arr = array.array('i', [1, 2, 3, 4])
arr[0] = 99          # mutable
arr.append(5)        # mutable
print(arr)           # array('i', [99, 2, 3, 4, 5])

# arr.append('x')    # TypeError – must be int
```

Q81. What is `ctypes` and how does it expose mutability at the C level?

`ctypes` allows direct manipulation of memory via C-compatible structures. You can create mutable C-level arrays and pass pointers to C functions — bypassing Python's normal object model.

```
import ctypes

# Mutable C integer array
IntArray = ctypes.c_int * 3
arr = IntArray(1, 2, 3)
arr[0] = 99
print(list(arr))    # [99, 2, 3]
```

Q82. What is `\_\_slots\_\_` memory advantage and its effect on mutability?

Without `\_\_slots\_\_`, each instance stores attributes in a `\_\_dict\_\_` (a hash table, ~200-400 bytes overhead). `\_\_slots\_\_` uses a fixed-size C array of pointers (~56 bytes), drastically reducing per-instance memory. Attributes are still mutable unless protected by descriptors.

```
import sys

class WithDict:
    def __init__(self, x): self.x = x

class WithSlots:
    __slots__ = ('x',)
    def __init__(self, x): self.x = x

print(sys.getsizeof(WithDict(1)))    # ~48 + dict overhead ~232
print(sys.getsizeof(WithSlots(1)))   # ~56 – much smaller
```

Q83. How does assignment to a slice work for lists?

Slice assignment replaces a sub-range of elements in-place. The replacement iterable does not need to be the same length.

```
a = [1, 2, 3, 4, 5]
a[1:3] = [20, 30, 40]    # replace 2 elements with 3
print(a)                 # [1, 20, 30, 40, 4, 5]
```

```
a[1:4] = []          # delete elements in-place
print(a)             # [1, 4, 5]

a[:] = [9, 9, 9]     # replace entire contents
print(a)             # [9, 9, 9]
```

Q84. What is `__setitem__` and how does it relate to mutability?

`__setitem__` is the dunder for `obj[key] = value`. Defining it on a class makes instances support item assignment. Immutable types either don't define it or raise `TypeError` inside it.

```
class FixedList:
    def __init__(self, data): self._data = list(data)
    def __getitem__(self, i): return self._data[i]
    def __setitem__(self, i, v):
        if not isinstance(v, int):
            raise TypeError('Only ints allowed')
        self._data[i] = v

fl = FixedList([1, 2, 3])
fl[0] = 99
# fl[0] = 'x'    # TypeError
print(fl[0])     # 99
```

Q85. How does `object.__new__` relate to immutability for built-in types?

For immutable types like `int`, `str`, `tuple`, the value must be set in `__new__` because `__init__` cannot change the object after creation.

```
class PositiveInt(int):
    def __new__(cls, value):
        if value <= 0:
            raise ValueError('Must be positive')
        return super().__new__(cls, value)

x = PositiveInt(5)
print(x + 1)    # 6 - behaves like int
# PositiveInt(-1) # ValueError
```

Q86. What is `MappingProxyType` and when is it useful?

`types.MappingProxyType` creates a read-only view of a dict. Useful for exposing class internals (like `__dict__` on a class) without allowing modification.

```
from types import MappingProxyType

original = {'a': 1, 'b': 2}
proxy = MappingProxyType(original)

print(proxy['a'])    # 1
```

```
# proxy['a'] = 99      # TypeError: 'mappingproxy' does not support assignment

original['c'] = 3
print(proxy['c'])      # 3 – proxy reflects changes to original
```

Q87. How does Python's `@property` enforce controlled mutability?

A property with only a getter makes the attribute effectively read-only. Adding a setter allows controlled writes with validation logic.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self): return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0: raise ValueError('Negative radius')
        self._radius = value

    @property
    def area(self):          # read-only computed
        import math
        return math.pi * self._radius ** 2

c = Circle(5)
c.radius = 10
# c.area = 99    # AttributeError – no setter
```

Q88. What is `ChainMap` and how does it relate to references?

`collections.ChainMap` wraps multiple dicts and provides a unified read/write view. Writes go to the first map; reads search maps in order. Inner dicts are NOT copied — they are referenced.

```
from collections import ChainMap

defaults = {'color': 'blue', 'size': 10}
overrides = {'color': 'red'}
merged = ChainMap(overrides, defaults)

print(merged['color'])    # red – from overrides
print(merged['size'])     # 10 – from defaults

merged['size'] = 20       # writes to overrides
print(defaults['size'])   # 10 – defaults unchanged
```

Q89. What is the difference between mutable and immutable class attributes in inheritance?

Mutable class attributes are shared across all subclasses and instances. An instance accessing a mutable class attribute gets the same object. Assigning to the attribute on an instance creates an instance attribute, shadowing the class one.

```
class Base:
    shared_list = []
    shared_int = 0

class Child(Base): pass

Child.shared_list.append(1)    # mutates Base.shared_list
print(Base.shared_list)       # [1]

child = Child()
child.shared_int = 99          # creates instance attr – shadows class
print(Child.shared_int)       # 0 – class attr unchanged
```

Q90. What is `\_\_iadd\_\_` and how does it relate to in-place mutation?

`\_\_iadd\_\_` implements `+=`. For mutable types it should mutate in-place and return `self`. If not defined, Python falls back to `\_\_add\_\_` and rebinds the variable to a new object.

```
class Stack:
    def __init__(self): self._data = []

    def __iadd__(self, item):
        self._data.append(item)
        return self          # MUST return self

    def __repr__(self): return f'Stack({self._data})'

s = Stack()
ref = s
s += 1
print(s is ref)    # True – in-place
print(s)           # Stack([1])
```

Q91. How does `functools.lru\_cache` interact with mutable arguments?

`lru\_cache` requires all arguments to be hashable (used as cache key). Passing mutable objects (lists, dicts) raises `TypeError`. Workaround: convert to immutable (tuple, frozenset) before calling.

```
from functools import lru_cache

@lru_cache(maxsize=128)
def process(data: tuple):    # tuple, not list
    return sum(data)

print(process((1, 2, 3)))    # 6
print(process((1, 2, 3)))    # 6 – cache hit
```

```
# process([1,2,3])           # TypeError: unhashable type: 'list'
```

Q92. What is ``dataclasses.field(default_factory=...)``?

When a dataclass field has a mutable default, you must use ``field(default_factory=list)`` instead of ``field(default=[])``. Each instance gets a fresh mutable object from the factory.

```
from dataclasses import dataclass, field

@dataclass
class Bag:
    items: list = field(default_factory=list)

b1 = Bag(); b2 = Bag()
b1.items.append(1)
print(b2.items)  # [] - independent

# @dataclass
# class BadBag:
#     items: list = []  # ValueError: mutable default
```

Q93. How do you safely pass a mutable config dict into a class?

Copy (shallow or deep) the dict in ``__init__`` to avoid the caller mutating the object's internal state after construction.

```
import copy

class Service:
    def __init__(self, config: dict):
        self._config = copy.deepcopy(config)  # own copy

cfg = {'host': 'localhost', 'ports': [80, 443]}
svc = Service(cfg)
cfg['ports'].append(8080)
print(svc._config['ports'])  # [80, 443] - unaffected
```

Q94. What is the 'observer pattern' and how does mutability enable it?

Observers hold a reference to the same mutable subject. When the subject mutates, all observers that hold a reference see the updated state without needing explicit notification for reads.

```
class DataStore:
    def __init__(self): self.data = []

store = DataStore()

class Observer:
    def __init__(self, store): self.store = store  # alias
    def view(self): return self.store.data
```

```
obs = Observer(store)
store.data.append(42)
print(obs.view())    # [42] - sees mutation through shared ref
```

Q95. What is `__delitem__` and how does it relate to mutability?

`__delitem__` is called for `del obj[key]`. Implementing it makes a class support item deletion — a mutable operation. Immutable containers do not define it.

```
class Registry:
    def __init__(self): self._store = {}
    def __setitem__(self, k, v): self._store[k] = v
    def __getitem__(self, k): return self._store[k]
    def __delitem__(self, k): del self._store[k]

r = Registry()
r['a'] = 1
del r['a']
# r['a']    # KeyError
```

Q96. How do `__enter__` and `__exit__` protect mutable state?

Context managers can acquire a lock on entry and release it on exit, ensuring that mutable shared state is accessed safely.

```
import threading

class SafeList:
    def __init__(self):
        self._data = []
        self._lock = threading.Lock()

    def __enter__(self):
        self._lock.acquire()
        return self._data

    def __exit__(self, *args):
        self._lock.release()

sl = SafeList()
with sl as data:
    data.append(1)    # thread-safe access
```

Q97. What is the difference between `id()` and `hash()`?

`id()` is unique per object lifetime (memory address in CPython) and works on any object. `hash()` is defined only for hashable objects and must be stable and consistent with `__eq__`. Two equal objects must have the same hash; different objects may share a hash (collision).

```

a = (1, 2)
b = (1, 2)

print(a is b)      # False – different objects
print(id(a) == id(b)) # False
print(hash(a) == hash(b)) # True – equal value, equal hash
print(a == b)      # True

```

Q98. How does Python's `array.array` compare to `list` for mutability and performance?

`array.array` is mutable and compact (stores C primitives, no boxing). List stores Python object pointers. For numeric data, `array` uses ~4-8x less memory and is faster for bulk operations. For general use, list is more flexible.

```

import array, sys

lst = list(range(10**4))
arr = array.array('l', range(10**4))

print(sys.getsizeof(lst)) # ~87624 bytes
print(sys.getsizeof(arr)) # ~81936 bytes (less overhead)

arr[0] = 99 # mutable, like list

```

Q99. What is a 'persistent data structure' and does Python have any?

A persistent data structure preserves previous versions on modification. Python's built-in types are not persistent. The `pyrsistent` library provides persistent (immutable) vectors, maps, and sets with structural sharing ($O(\log n)$ updates).

```

# pip install pyrsistent
from pyrsistent import pvector, pmap

v1 = pvector([1, 2, 3])
v2 = v1.append(4) # new version; v1 unchanged
print(v1)        # pvector([1, 2, 3])
print(v2)        # pvector([1, 2, 3, 4])

m1 = pmap({'a': 1})
m2 = m1.set('b', 2) # new version
print(m1)          # pmap({'a': 1})

```

Q100. Putting it all together — demonstrate the key mutability rules in one snippet.

This comprehensive example shows: aliasing, shallow copy pitfall, deep copy solution, mutable default trap, immutable workaround, and the pass-by-object-reference model.

```

import copy

```



```

# 1. Aliasing
a = [1, 2, 3]
b = a
b.append(4)
print('Alias:', a)           # [1, 2, 3, 4]

# 2. Shallow copy pitfall
nested = [[1], [2]]
shallow = nested.copy()
shallow[0].append(99)
print('Shallow:', nested)    # [[1, 99], [2]]

# 3. Deep copy solution
nested = [[1], [2]]
deep = copy.deepcopy(nested)
deep[0].append(99)
print('Deep:', nested)      # [[1], [2]] - safe

# 4. Mutable default trap & fix
def bad(lst=[]):
    lst.append(1); return lst
print(bad(), bad())         # [1] [1, 1] - shared default

def good(lst=None):
    if lst is None: lst = []
    lst.append(1); return lst
print(good(), good())       # [1] [1] - fresh each call

# 5. Pass-by-object-reference
def rebind(x): x = [99]
def mutate(x): x.append(99)
lst = [1, 2]
rebind(lst); print('Rebind:', lst) # [1, 2]
mutate(lst); print('Mutate:', lst) # [1, 2, 99]

```